

DIFFERENCE-AWARE RETRIEVAL POLICIES FOR IMITATION LEARNING

Anonymous authors
Anonymous submission

ABSTRACT

Parametric imitation learning via behavior cloning (BC) suffers from poor generalization to out-of-distribution states because errors compound during deployment. Our central claim is that this fragility can be reduced *without* leaving the standard BC setting, simply by reusing the training data at inference time. To this end we contribute **Difference-Aware Retrieval Policies for Imitation Learning (DARP)**, a semi-parametric, retrieval-based policy that makes three contributions. First, we reparameterize imitation learning in terms of local neighborhood structure rather than direct state-to-action mappings: DARP predicts actions from the k -nearest expert neighbors of a query state, their actions, and the relative distance vectors between neighbor and query states. Second, we show this reparameterization implicitly performs parameter-free Laplacian smoothing, and we prove it reduces policy variance and improves stability over BC while requiring no additional assumptions – no extra data collection, no online expert feedback, and no task-specific knowledge. Third, we demonstrate that these gains are broad and consistent: DARP improves over standard behavior cloning by 15–46% across continuous control and robotic manipulation, and across representations ranging from low-dimensional states to high-dimensional visual features. Code and demos will be released upon acceptance.

1 INTRODUCTION

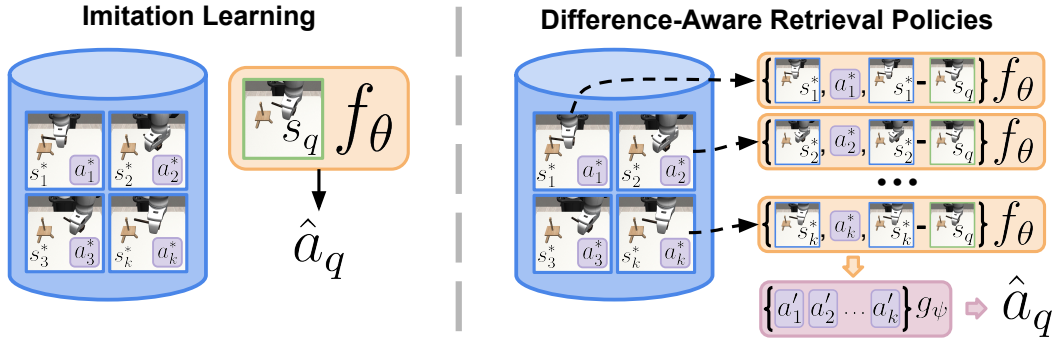


Figure 1: **Overview of DARP:** Unlike standard BC (left), DARP (right) utilizes a retrieval-based reparameterization centered around difference vectors between query states and retrieved neighbors. In standard behavior cloning, the dataset of expert state-action pairs is used only for training and is discarded at inference-time, while DARP utilizes it to perform retrieval to find a local neighborhood of expert state-action pairs around each query point s_q .

Imitation learning via behavior cloning (BC) (Pomerleau, 1991) has enabled robots to learn complex, dexterous behaviors from expert demonstrations (Zhao et al., 2023; Chi et al., 2024; Black et al., 2024; Chung et al., 2014). Yet despite its simplicity, BC often proves brittle in practice, especially for long-horizon tasks (Ross et al., 2011). The core issue is *covariate shift*: small errors accumulate during rollouts, driving the agent into states not well represented in the demonstration data (Spencer et al., 2021; Ross et al., 2011). In such out-of-distribution regions, BC policies are highly unstable,

producing unreliable and high-variance behavior that frequently leads to failure. Reducing this instability, without changing what the learner is allowed to assume, is the problem this paper sets out to solve.

This problem is well recognized, and many approaches have been proposed to mitigate compounding error (Ross et al., 2011; Venkatraman et al., 2015; Ke et al., 2024b; Levine et al., 2020). Crucially, however, these typically go beyond the standard BC assumptions, requiring simulators, interactive experts, large quantities of sub-optimal data, or strong task-specific structure. Our first commitment is therefore a constraint on the solution: we remain in the pure BC regime, learning only from expert state–action pairs with no additional supervision or feedback. This frames the central question of the paper: *can we reduce the variance of BC policies using only the original demonstration dataset?* Everything that follows is an answer to this question.

To motivate our answer, consider the problem from a statistical standpoint. BC minimizes only the supervised risk on expert states. This controls bias on the training distribution, but leaves variance unchecked: in low-density regions of the state space (which are often encountered during closed-loop rollouts), the learned policy can oscillate arbitrarily. A natural remedy is to enforce *smoothness*, so that nearby states yield similar predicted actions (Kobayashi, 2022; Asadi et al., 2018; Ke et al., 2024a; Chen et al., 2024). This discourages spurious fluctuations and improves rollout stability. Several approaches to encourage smoothness have been explored, see related work in Section 4.

Although sometimes effective, each has drawbacks that we build on rather than repeat: augmentation does not guarantee consistency, global priors can blur distinct behaviors, temporal penalties only act along time (not space), and explicit graph regularizers require tuning extra smoothness hyperparameters. A recurring theme is that smoothness is either bought with an extra hyperparameter or imposed globally.

The second thread we draw on contrasts global and local learning. “Global” supervised models (Black et al., 2024; Zhao et al., 2023; Chi et al., 2024) attempt to compress the entire demonstration dataset into a single parametric function, which is typically brittle under distribution shift. “Local” methods (Pari et al., 2022; Mansimov & Cho, 2018; Salzberg & Aha, 1994) instead adapt predictions to the structure of the dataset itself, consulting neighborhoods of similar states and generating outputs from non-parametric or semi-parametric operations on the training distribution of expert behavior. This locality offers robustness since it avoids reliance on a single parametric function, but it also has limitations: its effectiveness inherently depends on the distance metric, naive averaging of a neighborhood can blur distinct actions and struggle to represent multimodality, and treating neighbors only in terms of their absolute states can limit generalization. The gap we identify, and the one our method is designed to close, is a policy that inherits the robustness of local methods and the stability of regularized global learning while adding neither hyperparameters nor assumptions.

Our approach. We introduce **Difference-Aware Retrieval Policies for Imitation Learning (DARP)**, which occupies exactly this gap. At inference time, rather than predicting actions from the current query state alone via a feedforward pass on a parametric function, DARP (Fig. 1) first retrieves a set of k neighbors from the training corpus and then predicts k actions, each conditioned on a tuple of (neighbor state, associated action, and difference from the query state). These neighbor-informed predictions are aggregated in a permutation-invariant manner into a single robust action. The design does two things at once: it grounds predictions in observed data (via non-parametric retrieval) and implicitly enforces local consistency (via parametric prediction conditioned on the retrieved neighbors). Because this happens inside the architecture, the smoothing is free: neighbor aggregation approximates a Laplacian filter on the k -NN (k -nearest neighbor) graph of expert states, a parameter-free form of smoothing that adapts to the local density and geometry of the dataset, all while staying strictly within the standard BC assumptions.

Contributions. We support these claims with both theory and experiments. Concretely, this paper contributes:

- **A difference-aware retrieval reparameterization of imitation learning (DARP).** We recast BC as prediction over local neighborhood structure, conditioning on retrieved neighbors and, critically, their difference vectors to the query state, then aggregating permutation-invariantly.

- **A theoretical account of why it works.** We formalize the connection to Laplacian regularization and prove that DARP’s neighbor aggregation implicitly applies a fixed low-pass spectral filter, yielding provably lower variance and better stability than BC without any extra hyperparameter.
- **Broad empirical gains under the same assumptions as BC.** We show 15–46% improvements over standard behavior cloning across continuous control (MuJoCo), robotic manipulation (Robosuite, RoboCasa), and high-dimensional visual imitation (Robosuite with image state), demonstrating that DARP is a general, scalable architecture compatible with rich policy classes such as transformers and Gaussian mixture models.
- **Ablations that isolate what matters.** A careful ablation study identifies the difference-vector representation and permutation-invariant aggregation as the decisive components, yielding general-purpose insights into retrieval-based algorithms for sequential decision-making.

2 DIFFERENCE-AWARE RETRIEVAL POLICIES FOR IMITATION LEARNING

This section develops our central methodological contribution: a class of imitation learning methods that get the best of both “global” parametric learning and “local” learning. The contribution is a new architecture paired with a simple training objective that learns under exactly the same requirements as typical behavior cloning, yet delivers significant improvements both theoretically and empirically. We derive the guarantees from first principles so the source of the improvement is transparent. Readers primarily interested in the practical algorithm may skip to Section 2.5 for a self-contained summary of DARP before proceeding to Section 3 for empirical results.

We build the contribution in three steps, each isolating one idea. As a warmup, we define the problem setting (Section 2.1) and introduce a variant of regularized imitation learning (Section 2.2) that imposes additional structure from the data to improve variance, generalization, and stability. In Section 2.3 we then make our key observation: the benefits of explicit regularization can be obtained *implicitly* by modifying the policy *architecture* rather than the objective. Finally, Section 2.4 turns this observation into our practical algorithm DARP, a semi-parametric retrieval-augmented architecture that applies broadly to imitation learning with modern neural networks and generative modeling tools.

2.1 PRELIMINARIES: BEHAVIOR CLONING FOR IMITATION LEARNING

We operate in the typical imitation learning setting, formalized by a finite-horizon Markov Decision Process (MDP), $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, P_0\}$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and P_0 is the initial state distribution. A *policy* maps a state to a distribution of actions $f_\theta : \mathcal{S} \rightarrow \Delta_{\mathcal{A}}$ so as to maximize task-relevant objectives (for brevity, we drop the subscript θ when discussing the general function class f). We assume access to expert human-provided demonstrations $\mathcal{D}^*$ as a collection of state-action pairs: $\mathcal{D}^* = \{(s_j^*, a_j^*)\}$. We use the notation s^* and a^* specifically to denote states and actions belonging to the expert dataset. The behavior cloning (Pomerleau, 1991) algorithm learns a policy f_θ from this dataset by casting imitation as a typical supervised learning problem — $\arg \max_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\log(f_\theta(a^* | s^*))]$. While the distribution class of f_θ can be an arbitrary complex generative model (Lipman et al., 2023; Chi et al., 2024), we will start with a Gaussian parameterization for the sake of simplicity¹.

2.2 WARM-UP: NEIGHBOR MANIFOLD REGULARIZED IMITATION LEARNING

The starting point for our first step is a known weakness of BC. While behavior cloning minimizes only the supervised imitation loss over expert states drawn from $\mathcal{D}^*$, such an objective alone does not control how the policy behaves on states that deviate from the manifold of expert states. In practice, accumulating errors lead the agent to out-of-distribution regions where a BC policy may act arbitrarily, especially for overparameterized neural networks (Ross & Bagnell, 2010).

Our observation is that behavior cloning enforces function evaluations only at the training states; it never accounts for the relationship between states (and their corresponding actions) within a

¹We show that this can be relaxed in Section 2.4

neighborhood, and thus ignores the underlying data manifold. To inject this information into policy learning, we consider a modified objective with a regularization term that explicitly encourages *local consistency* of predictions: nearby states in the expert dataset should map to similar actions. This intuition yields the neighborhood-regularized loss ($\mathcal{L}_{\text{MRIL}}$) below, in which the standard imitation objective ($\mathcal{L}_{\text{BC}}$) is combined with a smoothness penalty ($\mathcal{L}_S$) that forces predictions to respect the geometry of the dataset rather than relying on pointwise supervision alone.

$$\mathcal{L}_{\text{MRIL}}(f) = \underbrace{\mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)]}_{\text{supervised risk}(\mathcal{L}_{\text{BC}})} + \lambda \underbrace{\mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right]}_{\text{smoothness regularizer}(\mathcal{L}_S)}, \quad (1)$$

where $\ell(f(s^*), a^*)$ is the supervised imitation loss, $\mathcal{N}_k(s^*)$ are the k -nearest neighbors of s^* from the expert dataset, and the weights $w_i(s^*)$ are normalized kernel weights based on the state differences — $w_i(s^*) \propto K_\Delta\left(\frac{\|s_i^* - s^*\|}{h}\right)$. As we sketch below (and prove in detail in Appendix A.1.1), this penalty is exactly a form of *manifold regularization*, or Laplacian smoothing, that penalizes the policy for high-frequency variation across the neighborhood of expert states. This is the first payoff of the section: manifold regularization provably improves policy variance, stability, and generalization, as the following theorem makes precise.

Theorem 1 (Manifold Regularized BC ($\mathcal{L}_{\text{MRIL}}$) improves over vanilla BC ($\mathcal{L}_{\text{BC}}$)). *Let $f^* : \mathcal{S} \rightarrow \mathcal{A}$ be the true, underlying expert policy, assumed to be C^2 -smooth on a compact state space $\mathcal{S}$. Let $f : \mathcal{S} \rightarrow \mathcal{A}$ denote the learned policy estimator. Consider two estimators trained on expert demonstrations:*

1. **Vanilla BC:** a global supervised model minimizing

$$\mathcal{L}_{\text{BC}}(f) = \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)].$$

2. **MRIL:** a neighbor-based estimator minimizing

$$\mathcal{L}_{\text{MRIL}}(f) = \mathcal{L}_{\text{BC}}(f) + \lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right],$$

where $w_i(s^*)$ are the kernel weights defined above and $\lambda > 0$.

Then, under the smoothness assumption on f , the following hold:

- (i) Variance reduction: The Laplacian penalty in MRIL acts as a data-dependent Tikhonov regularizer, yielding smaller estimator variance than vanilla BC.
- (ii) Smoothness guarantee: Minimizers of $\mathcal{L}_{\text{MRIL}}$ satisfy a uniform bound on the local Lipschitz constant of f , whereas vanilla BC admits interpolants with arbitrarily large Lipschitz constants between training states.
- (iii) Policy stability: In a closed-loop rollout, the deviation recursion

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|^2$$

accumulates error linearly for vanilla BC, but sublinearly for MRIL, since the smoothness regularizer enforces $\|f(s^*) - f(s'^*)\| = O(\|s^* - s'^*\|)$ for neighbors s^*, s'^* .

This suggests that MRIL enjoys strictly better generalization and stability guarantees than BC.

Proof sketch. We defer the detailed proof to Appendix A.1.1, but provide a brief sketch. The key idea of the proof is to first show that the smoothness regularizer directly corresponds to a graph Laplacian penalty on a graph constructed by a k -nearest neighbor (k -NN) affinity matrix defined by the kernel w_i . Next, we show that as the number of samples tends to infinity, this graph Laplacian penalty converges to the weighted Dirichlet energy (Belkin & Niyogi, 2008; Zhou et al., 2003). Minimizing this Dirichlet energy (1) ensures that the learned f is locally Lipschitz almost everywhere, ensuring smoothness and, in turn, policy stability, and (2) corresponds to Tikhonov regularization, thereby reducing estimator variance, while keeping the bias controlled. $\square$

² L_s and L_a are the Lipschitz constants of the environment transition dynamics with respect to state and action, respectively.

The takeaway is that the smoothness regularizer does more than penalize pairwise disagreements between neighbors: it drives the learned policy to be smooth with respect to the underlying data manifold. In particular, it shrinks the local Lipschitz constant of f along directions where the data density $p(s)$ is high, so that small changes in state produce small, consistent changes in the predicted action. As a result, the policy generalizes more reliably on in-distribution (ID) states and extrapolates in a structured manner on new out-of-distribution (OOD) states within the neighborhood. This establishes the target behavior; the remaining question, taken up next, is how to obtain it without paying for an explicit regularizer.

2.3 IMPLICIT MANIFOLD REGULARIZATION VIA IN-CONTEXT ARCHITECTURES

While the MRIL objective amortizes local learning and improves over vanilla BC, it carries two drawbacks that motivate our second and central step. First, it introduces a hyperparameter λ that must be tuned to balance supervised accuracy against smoothness. Second, optimizing a modified, regularized objective rather than a standard BC objective can distort the optimization landscape in adverse ways. This raises the question that drives the rest of the method: *can we obtain the same benefits as MRIL (Eq. 1) by changing the policy architecture rather than the objective?*

Our answer, and the key methodological contribution of this section, is yes: we introduce a retrieval-based architectural change that produces an *implicit* manifold regularization effect (iMRIL) while training on a standard imitation objective. iMRIL recovers the benefits of Laplacian smoothing from MRIL (illustrated in Fig. 2) without ever introducing λ as a training hyperparameter. Section 2.4 then builds on iMRIL to develop our practical algorithm, DARP.

iMRIL architecture: The idea behind iMRIL is deliberately simple, and this simplicity is the point: we move the neighborhood aggregation (averaging) operation out of the objective (as in Eq. 1) and into the architecture itself. Instead of learning a standard feedforward predictor $f(s)$ trained against a neighborhood-regularized smoothness objective (Eq. 1), we embed the structure of neighborhood aggregation directly into the parameterization of the action predictor $\hat{f}$, keeping the objective standard imitation learning. Concretely, iMRIL learns the parameters of a per-state predictor f_θ so that the aggregated predictor $\hat{f}(s^*) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s^*)} f_\theta(s_i^*)$, formed over nearest-neighbor states from the training set $\{s_i^*\}_{i \in \mathcal{N}_k(s^*)}$, accurately predicts the corresponding expert action a^* . With this parameterization, iMRIL optimizes at training time:

$$\arg \min_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} \left[\left\| \underbrace{\left(\frac{1}{k} \sum_{i \in \mathcal{N}_k(s^*)} f_\theta(s_i^*) \right)}_{\hat{f}(s^*)} - a^* \right\|_2 \right] \quad (2)$$

At deployment time, inference on a new state s_q is performed simply by retrieving the k -NN of s_q from the training set and applying the same neighborhood aggregation $\hat{a} = \hat{f}(s_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} f_\theta(s_i^*)$. As Section 2.4 shows, the specific parameterization of f is crucial to the empirical performance of iMRIL, and refining it is precisely what leads to DARP.

The intuition is worth stating plainly: we parameterize the action predictor $\hat{f}$ as an aggregation of neighbor-state predictions $f(s_i^*)$ and then learn f . Because we supervise the post-aggregation function, no individual f prediction can be arbitrarily non-smooth, which is exactly what confers the benefits of Section 2.2. We make this connection rigorous next, proving a direct equivalence between iMRIL and the Laplacian regularization of Section 2.2.

Equivalence between iMRIL and MRIL: While we defer a full proof of formal equivalence between MRIL and iMRIL to the Appendix Section A.1, we state our main result and a proof sketch to this effect here.

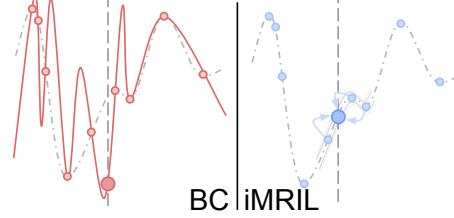


Figure 2: **iMRIL implicitly achieves Laplacian smoothing, which reduces variance and enforces local consistency**, whereas the lack of smoothness constraint on standard BC allows for arbitrarily jagged function approximations.

Theorem 2 (iMRIL is parameter-free Laplacian regularization for BC (MRIL)). *Consider the symmetric normalized k -NN graph Laplacian L (defined in Section 2.2), with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$.*

The minimizers of the explicit MRIL objective (Section 2.2) and the implicit iMRIL objective (Section 2.3) have the following closed form expansions

$$f_{\text{MRIL}} = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j \quad \hat{f}_{\text{iMRIL}} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j$$

iMRIL's neighbor aggregation step applies the fixed spectral filter $\phi_{\text{iMRIL}}(\mu) = 1 - \mu$ to the graph Laplacian L , preserving low-frequency modes and suppressing high-frequency modes. The congruence between $\hat{f}_{\text{iMRIL}}$ and f_{MRIL} shows that iMRIL is equivalent to a built-in form of Laplacian smoothing (MRIL) with effective $\lambda \approx 1$ in normalized units. Unlike explicit regularization, this implicit filter requires no additional hyperparameter tuning.

Proof sketch. We defer full details to Appendix Section A.1.2. The explicit regularizer admits a spectral solution by diagonalizing the k -NN Laplacian, yielding a filter of the form $(1 + \lambda \mu)^{-1}$ on each eigenmode. The implicit objective can be expressed as neighbor aggregation $\hat{f} = S f$ with $S = D^{-1} A$, the random-walk matrix, which has the same eigenvectors and applies the fixed filter $1 - \mu$. Intuitively, both act as low-pass filters on the graph: modes with small eigenvalues (smooth variation across the data manifold) are largely preserved, while modes with large eigenvalues (rapid, high-variance fluctuations between neighbors) are strongly damped. Thus iMRIL implicitly performs Laplacian smoothing, reducing variance and enforcing local consistency without needing to tune λ . $\square$

One clarification guards against a misreading: the implicit Laplacian smoothing view does not replace the need to learn a policy; it constrains the class of functions representable after aggregation. The neighbor-conditioned network f_θ learns how expert actions vary under local perturbations, proposing a locally adapted action for each neighbor, while the aggregation operator reduces variance by smoothing these proposals across the neighborhood. This division of labor is the crux of the contribution: learning supplies accuracy by correcting local bias, and aggregation supplies stability by controlling variance.

2.4 DIFFERENCE-AWARE RETRIEVAL POLICIES: A PRACTICAL INSTANTIATION OF iMRIL FOR IMITATION LEARNING

With the conceptual framework of iMRIL in place, we now deliver the practical algorithm that is this section's headline contribution. Building on the objective in Eq. 2, DARP makes two deliberate design choices, each of which we later show is essential: (1) a difference-based parameterization of f_θ , and (2) a permutation-invariant neighbor aggregation. Together these are what turn the theory into strong empirical performance.

2.4.1 DIFFERENCE-BASED PARAMETERIZATION OF f_θ

The objective in Eq. 2 leaves the parameterization and input representation of f_θ open. Our key design decision follows from a single observation: since neighborhood aggregation must learn how expert actions vary under local perturbations, f_θ should be given the *difference* between a query state and each neighbor state so it can propose a locally adapted action per neighbor. This is the “difference-aware” idea from which DARP takes its name. Concretely, in **Difference-Aware Retrieval Policies (DARP)**, rather than parameterizing f_θ by $f_\theta(s_i^*)$ alone, we additionally supply the optimal neighbor action a_i^* and the difference vector $\Delta s_i = s_i^* - s_q$: the predictor produces an action candidate a'_i for query state s_q and neighbor (s_i^*, a_i^*) as $a'_i = f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)$.

Let $\mathcal{N}_k(s_q)$ be the index set of the k -nearest neighbors retrieved according to some distance function $d(s_q, s_i^*)$.³ For generating predictions with DARP, we can then perform neighborhood aggregation (as outlined in Section 2.3) to predict an action for any query state s_q

$$\hat{a}_q = f_{\text{DARP}}(s_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} a'_i \quad (3)$$

$$= \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} f_{\theta}(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q). \quad (4)$$

At training time, this can be used to define a straightforward imitation learning objective from the expert dataset $\mathcal{D}^*$:

$$\arg \min_{\theta} \mathbb{E}_{(s_q^*, a_q^*) \sim \mathcal{D}^*} [\|f_{\text{DARP}}(s_q^*) - a_q^*\|^2] \quad (5)$$

where we optimize the parameters of the predictor f_{θ} to minimize the discrepancy between the predicted action $\hat{a}_q$ and the optimal action a_q^* . Because the objective stays this simple, f_{θ} can be any standard network – in our case, feedforward or convolutional networks – and no new machinery is needed to train it. We emphasize that the difference-based parameterization is not a cosmetic choice: our ablations in Section 3 show it is crucial for performance. At inference time, we generate actions by retrieving the k -NN and applying the neighborhood aggregation of Eq. 3.

2.4.2 GOING BEYOND LINEAR AGGREGATION

Our second design choice concerns aggregation, and here the contribution is a generalization that broadens DARP’s reach. So far aggregation has been simple averaging, $\hat{a}_q = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} a'_i$, but this is only a special case of the broader class of permutation-invariant aggregation functions $g_{\psi}(\{a'_i\}_{i \in \mathcal{N}_k(s_q)})$. In particular, g_{ψ} can be an expressive set-compliant model such as a set transformer (Lee et al., 2019) or DeepSets (Zaheer et al., 2017), generalizing Eq. 3 to $\hat{a}_q = g_{\psi}(\{f_{\theta}(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)\}_{i \in \mathcal{N}_k(s_q)})$. Beyond added expressivity, a parametric aggregator unlocks a capability that plain averaging lacks: the representation of richer, multimodal action distributions (e.g., Gaussian mixture models (Pignat & Calinon, 2019) or diffusion models (Chi et al., 2024)) instead of the single Gaussian implicit in the L_2 -regression objective of Eq. 5. Rather than predicting $\hat{a}_q$ directly, DARP can predict the parameters α of an action distribution $p(a_q; \alpha)$ – e.g., the means, covariances, and weights of a Gaussian mixture, or the score function of a diffusion model – enabling maximum-likelihood training of multimodal action distributions in place of unimodal L_2 -regression:

$$\arg \max_{\theta} \mathbb{E}_{(s_q^*, a_q^*) \sim \mathcal{D}^*} [\log p(a_q^*; \alpha_{\theta}(s_q^*))], \text{ where } \alpha_{\theta}(s_q^*) = g_{\psi}(\{f_{\theta}(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q^*)\}_{i \in \mathcal{N}_k(s_q^*)}) \quad (6)$$

Inference for a query state s_q can be performed by sampling $\hat{a}_q \sim p(\cdot; \alpha_{\theta}(s_q))$, constructing $\alpha_{\theta}(s_q) = g_{\psi}(\{f_{\theta}(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)\}_{i \in \mathcal{N}_k(s_q)})$ from a set of neighbors retrieved at test time. We refer readers to Appendix Section A.3 for detailed training pseudocode.

2.5 SUMMARY

To recap the contributions of this section: (i) we show that encouraging local consistency when fitting a BC policy is exactly manifold regularization (Laplacian smoothing), which provably improves variance, smoothness, and stability over standard BC (Theorem 1); and (ii) we show this smoothing can be induced *implicitly*, with no explicit regularizer and no hyperparameter λ , by building neighborhood aggregation into the policy architecture (Theorem 2). DARP is the concrete algorithm these results motivate. At training time, for each expert state-action pair (s_q^*, a_q^*) , DARP retrieves the k -nearest neighbor states from $\mathcal{D}^*$, computes the difference vectors $\Delta s_i = s_i^* - s_q^*$ from each neighbor to the query, passes each neighbor tuple $(s_i^*, a_i^*, \Delta s_i)$ through a network f_{θ} to produce candidate

³In our work we use the Euclidean distance in a pre-trained embedding space, although other neighborhood functions are also applicable. We refer the reader to Appendix Section A.2.1 for a thorough discussion of design decisions in constructing neighborhood sets via retrieval.

actions, and aggregates them with a permutation-invariant function g_ψ to predict the final action – all trained with nothing more than the standard imitation learning objective. This is what we evaluate next.

3 EXPERIMENTAL EVALUATION

Our experiments are organized to test each of the paper’s claimed contributions directly, through three questions: **Q1:** Does DARP consistently outperform standard behavior cloning? (testing the broad-gains contribution), **Q2:** Does it handle more complex state representations and action distributions? (testing generality), and **Q3:** Which architectural components drive the gains? (testing that the difference-vector and permutation-invariant design choices are the decisive ones). To answer them, we evaluate across multiple domains spanning low-dimensional states, high-dimensional image features, and diverse action representations: continuous control (MuJoCo), robotic manipulation (Robosuite, RoboCasa), and purpose-built discontinuous environments that stress-test the neighbor-based approach. The results below confirm all three claims.

3.1 BASELINE COMPARISONS AND TASK DESCRIPTIONS

MuJoCo Tasks: The MuJoCo (Todorov et al., 2012; Fu et al., 2020) tasks require controlling legged bodies of several embodiments to achieve forward locomotion on a flat plane: Hopper (single-legged hopping robot), Walker (bipedal humanoid), Ant (quadruped), and HalfCheetah (biped).

Robosuite Tasks: The Robosuite (Zhu et al., 2020) tasks each involve a single robotic arm manipulating objects: Stack (place a smaller cube on top of a larger one), Threading (guide a thin, needle-like tool into a small ring), and Square Peg (place a square wooden block with a central hole onto a square peg).

RoboCasa Tasks: The RoboCasa (Nasiriany et al., 2024) tasks each involve a single robotic arm manipulating objects in a randomized kitchen: Drawer (close an open drawer), Door (close an open microwave oven door), and Stove (twist a knob to turn off a stove burner).

Baseline Comparisons: To situate DARP’s gains, we compare against baselines that span the non-parametric, parametric, and in-context families it draws from: (1) **R&P (Sridhar et al., 2025):** directly executing the action of the nearest neighbor, (2) **LWR (Pari et al., 2022):** locally weighted regression on retrieved neighbors, (3) **BC:** standard parametric behavior cloning, our primary point of comparison, (4) **REGENT (Sridhar et al., 2025):** a transformer-based in-context learning method conditioned on retrieved neighbors, and (5) **MRIL:** the explicitly smoothed variant from Section 2.2, included so we can separate the value of implicit smoothing from explicit regularization.

3.2 CAN DARP CONSISTENTLY OUTPERFORM STANDARD BEHAVIOR CLONING? (Q1)

We first test the core contribution – that DARP improves over BC – on tasks with low-dimensional state representations, where distance metrics between states are well-defined and interpretable. This evaluation spans locomotion tasks from MuJoCo and robotic manipulation tasks from Robosuite and RoboCasa with data generated by MimicGen (Mandlekar et al., 2023). To isolate the contribution of the reparameterization itself, we deliberately keep the aggregation function g as a simple average of the neighbor action predictions a' here.

Method	Hopper	Ant	Walker	HalfCheetah
R&P (Sridhar et al., 2025)	711.82 $\pm$ 85.63	-305.97 $\pm$ 76.42	419.18 $\pm$ 50.21	-178.64 $\pm$ 29.75
LWR (Pari et al., 2022)	1703.78 $\pm$ 245.95	846.59 $\pm$ 216.06	1484.91 $\pm$ 356.54	1945.82 $\pm$ 567.26
BC	2313.65 $\pm$ 203.75	2376.20 $\pm$ 339.43	2658.40 $\pm$ 274.08	1063.23 $\pm$ 371.08
REGENT (Sridhar et al., 2025)	1819.39 $\pm$ 186.24	-302.10 $\pm$ 146.67	507.01 $\pm$ 76.10	169.85 $\pm$ 63.10
MRIL	2793.63 $\pm$ 156.41	3869.08 $\pm$ 241.00	4370.96 $\pm$ 168.13	701.58 $\pm$ 195.08
DARP	3545.57 $\pm$ 3.54	4383.28 $\pm$ 266.37	4894.01 $\pm$ 75.12	5515.41 $\pm$ 841.33
DARP Set Transformer	2965.86 $\pm$ 103.08	4063.79 $\pm$ 218.80	4752.42 $\pm$ 109.23	3417.85 $\pm$ 764.57

Table 1: **Q1 confirmed: both DARP and DARP Set Transformer outperform every baseline across all MuJoCo domains.** Comparison of DARP against BC and other baselines on MuJoCo environments using low-dimensional state. Scores are averaged over 100 independent trials with 95% confidence intervals; best results per column are in bold. The parametric policies use Multi-Layer Perceptrons; for results with diffusion policies, see Section 8.

The answer to Q1 is unambiguous: DARP improves substantially over standard behavior cloning in every tested environment. Gains range from 15–25 percentage points on robotic manipulation tasks and are large in absolute score on locomotion tasks (Table 1 and Table 2). Two comparisons sharpen the takeaway. First, purely non-parametric methods (R&P and LWR) perform poorly, so retrieval alone is not what drives DARP. Second, while MRIL almost always beats vanilla BC – corroborating the theory that smoothing helps – the top score on every task belongs to the full DARP architecture, showing that the implicit, difference-aware formulation delivers more than explicit smoothing does.

Having established the gain in the low-dimensional regime, we now stress the generality claim by asking whether the practical instantiation of DARP scales to higher-dimensional inputs such as images.

3.3 CAN DARP HANDLE MORE COMPLEX STATE REPRESENTATION AND ACTION DISTRIBUTIONS? (Q2)

High-Dimensional Visual Input Representations. The first half of Q2 asks whether the neighbor-based approach survives outside compact, low-dimensional

Method	Stack	Thrd.	Peg
BC	44	38	17
DARP	75	76	52

Table 3: **DARP scales to vision:** success rates (%) on vision-based Robosuite tasks, where DARP outperforms BC in all cases.



Figure 3: The “DrawerClose” RoboCasa task: the robotic arm must close the open drawer.



Figure 4: The “Threading” Robosuite task: the robotic arm must insert the needle implement into the small hole.

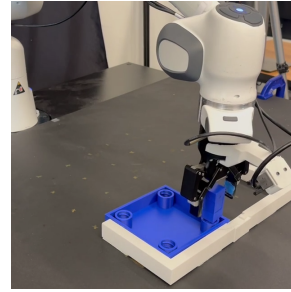


Figure 5: Real-world FurnitureBench square-table assembly task: the robot must pick up a table leg and screw it into a hole in the corner of the table-top.

Method	RoboCasa			Robosuite			Real
	Drawer	Door	Stove	Stack	Thrd.	Peg	Sq. Table
BC	54	29	28	47	37	46	44
DARP	85	45	43	72	63	62	92

Table 2: **DARP outperforms BC on every task across all three evaluation environments**, using low-dimensional state features. Scores are success percentages (RoboCasa & Robosuite: 100 trials; Real: 50 trials). The gain holds everywhere and is largest in the real world, where DARP more than doubles BC’s score – direct evidence for the broad-gains contribution. RoboCasa and Robosuite policies use Multi-Layer Perceptrons; the real-world results use diffusion policy, see Section 8.

states. We test this on simulated robotic manipulation with R3M image embeddings (Nair et al., 2022), where states are high-dimensional feature vectors extracted from visual observations (Table 3). The result strengthens the generality claim: not only does DARP beat standard BC, its average improvement ($\sim 35\%$) is actually *larger* here than on the same Robosuite tasks in low-dimensional state ($\sim 22\%$). In other words, DARP adapts to complex, high-dimensional representations even more effectively than to simple ones.

Multi-modal Action Distributions. The second half of Q2 concerns action distributions. Here the parametric-aggregator contribution pays off: on the multimodal Push-T environment, DARP solves the task over 20% better than behavior cloning, confirming that the generalized aggregation of DARP extends it to multimodal action modeling. We defer details to Appendix A.2.2.

3.4 HOW DO DIFFERENT ARCHITECTURAL COMPONENTS CONTRIBUTE TO DARP’S PERFORMANCE GAINS? (Q3)

Ablation Study: Q3 asks which design choices are actually responsible for the gains, directly testing the paper’s claim that the difference-vector representation and permutation-invariant aggregation are the decisive components. We conduct a systematic ablation in which each choice is removed or altered in turn: (1) standard DARP; (2) DARP without neighbor actions; (3) an ensemble of 10 BC agents; (4) DARP with random neighbors instead of distance-based selection; (5) DARP using the L_2 norm of the distance vector; (6) the BC baseline, which uses only the query state s_q ; (7) DARP conditioned on the query state instead of the query-to-neighbor difference vector; and (8) DARP with a permutation-*dependent* (i.e., **not** permutation-invariant) aggregator over the a ’s. Figure 6 reports the results.

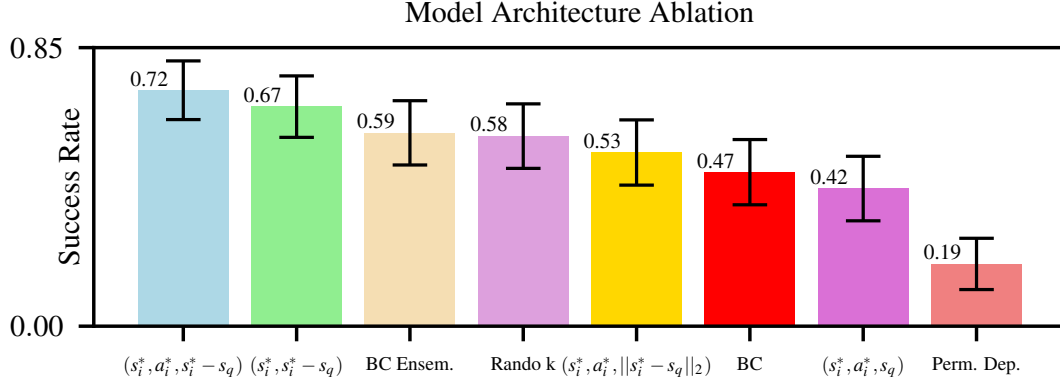


Figure 6: **Q3 answered: distance vectors and permutation invariance are the components that make DARP work.** Perturbing each core design choice shows that DARP’s success is most attributable to the difference-vector representation $(s_i^*, a_i^*, s_i^* - s_q)$, with permutation-invariant aggregation a close second. Success rate is averaged over 100 trials on the Robosuite Stack environment with 95% confidence intervals.

The ablation supports the contribution precisely as claimed. The two design choices we singled out in Section 2.4 – the difference-vector representation and permutation-invariant aggregation – are exactly the ones whose removal hurts most, while neighbor actions have a more modest effect. Random neighbor selection performs poorly, confirming that a meaningful distance metric is needed to form the neighborhood, and swapping the permutation-invariant aggregator g for a permutation-dependent one significantly degrades performance. In short, the components we highlighted as central are empirically the ones that matter.

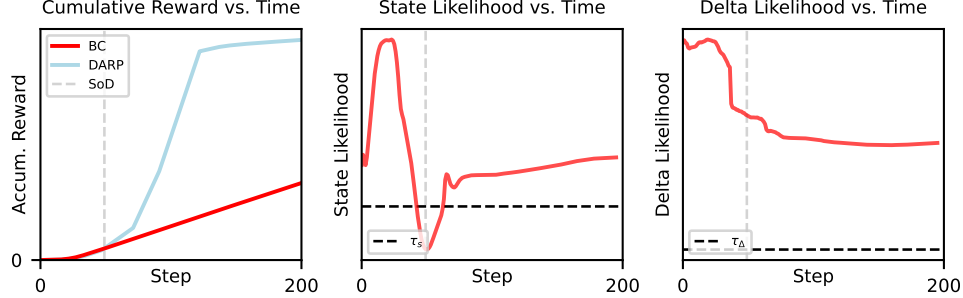


Figure 7: **Why DARP succeeds where BC fails: cumulative rewards on the Robosuite Stack task.** The two rollouts are initially identical and diverge only once BC drifts off-distribution – BC then fails while DARP succeeds. The vertical dashed line marks the divergence step (“SoD”). Crucially, at the SoD the state likelihood is $< \tau_s$ (OOD) while the difference-vector likelihood is $> \tau_\Delta$ (in distribution), pinpointing the difference-based reparameterization as the source of the recovery.

Divergence Analysis: Beyond the ablations, this analysis offers a mechanistic explanation for the contribution, tracing DARP’s advantage back to the difference-based reparameterization. We study rollouts where BC fails but DARP succeeds, identifying the “step of divergence” (SoD) as the point at which the two policies begin to receive significantly different reward. We set τ_s and τ_Δ to the 1st percentile of training-set likelihoods (i.e., 1% of training-time deltas are less likely than τ_Δ). The finding is consistent and striking: in all six rollouts across two tasks (Robosuite Stack and MuJoCo Hopper), the query state at the SoD has state likelihood $< \tau_s$ yet difference-vector likelihood $> \tau_\Delta$. That is, the state is out-of-distribution but its difference vector is not – exactly the regime in which DARP’s reparameterization should help, and precisely where it does. This directly supports our claim that DARP’s gains stem in part from better prediction on slightly out-of-distribution states (see Figure 7 for reward drift, SoDs, and likelihoods on one task). Appendix A.2.4 reports additional robustness experiments.

4 RELATED WORK

Non-Parametric Imitation Learning Methods: Non-parametric IL algorithms achieve surprising performance by leveraging local structure. VINN Pari et al. (2022) applies locally weighted regression to imitation, with strong results in image embedding spaces, and MiDiGaP von Hartz et al. (2025) uses mixtures of Gaussian processes to model multimodal trajectories and generalize rapidly. SEABO Lyu et al. (2024) uses retrieval for offline RL, rewarding transitions that lie close to neighbors. FlowRetrieval Lin et al. (2024), STRAP Memmel et al. (2025), and Behavior Retrieval Du et al. (2023) perform non-parametric retrieval and finetuning from large unlabeled datasets, generalizing via test-time training. The distinguishing contribution of DARP relative to all of these is its representation: it encodes each retrieved neighbor as a difference-aware tuple $(s_i^*, a_i^*, s_i^* - s_q)$ and learns a semi-parametric policy over them, rather than relying on purely non-parametric aggregation or test-time training. This is what lets DARP combine the variance reduction of local methods with the generalization of parametric policies.

Smoothness-Constrained Policy Learning: A large body of work enforces *explicit* smoothness constraints to improve policy stability and robustness. L2C2 Kobayashi (2022) studies model-free RL under local Lipschitz-continuity constraints, achieving smoothness and noise robustness without sacrificing expressiveness, and Asadi et al. (2018) take a similar route for model-based RL. CCIL Ke et al. (2024a) extends these ideas to generate synthetic corrective labels for imitation learning via a Lipschitz-constrained dynamics model, an approach later scaled to humanoid controllers Chen et al. (2024) to reduce deployment shakiness. The key contrast is that these methods buy smoothness with an explicit constraint or objective, whereas DARP’s contribution is to obtain the same smoothing *implicitly*, through an architectural change, while training on a standard imitation objective with no added hyperparameter.

In-Context Learning Methods: A parallel line studies retrieval through the lens of in-context imitation learning. REGENT Sridhar et al. (2025) feeds retrieved states, actions, and rewards into a causal transformer for retrieval-augmented generalization, and DPT Lee et al. (2023) pretrains transformers to predict actions from a query state and an in-context dataset, effectively learning to explore. Other in-context architectures include ICRT Fu et al. (2024), Instant Policy Vosylius & Johns (2025), and KAT Di Palo & Johns (2024). These methods target rapid adaptation to new tasks and environments; DARP instead targets a different contribution – higher performance and stability on standard imitation learning – and does so with a lightweight aggregation rather than a large in-context model.

5 CONCLUSION

We introduced **D**ifference-Aware **R**etrieval Policies for Imitation Learning (**DARP**) (DARP), a nearest-neighbor-based algorithm that reparameterizes imitation learning in terms of relative differences between query states and their nearest neighbors rather than learning direct state-to-action mappings. To restate the paper’s contributions and how each was borne out: (1) *a difference-aware retrieval reparameterization* that stays within the standard BC assumptions; (2) *a theoretical account* showing this reparameterization implicitly achieves parameter-free Laplacian smoothing, which we prove reduces variance and improves stability; and (3) *broad empirical validation* across diverse domains. Our experiments confirm three corresponding claims: DARP consistently outperforms standard behavior cloning with low-dimensional state representations; it retains its advantage across different state representations, action-distribution modeling requirements, and task complexities, with 15–46% improvements across the tested scenarios; and architectural ablations pinpoint difference vectors and permutation-invariant aggregation as the components responsible for the gains. Together these results establish DARP as a simple, assumption-light, and broadly applicable way to make imitation learning more robust to distribution shift.

6 REPRODUCIBILITY STATEMENT

Supplementary source code, policy and environment configuration files, MuJoCo experiment data, scripts for generating expert demonstrations for Robosuite tasks via MimicGen, and modality transformation code will be released upon acceptance. Results will be identical to those in the paper on

NVIDIA L40 and L40s GPUs, with the exception of results that require the use of a transformer (REGENT, Set Transformer), which are non-deterministic and may differ slightly from reported numbers.

REFERENCES

- Kavosh Asadi, Dipendra Misra, and Michael Littman. Lipschitz continuity in model-based reinforcement learning. In *International Conference on Machine Learning*. PMLR, PMLR, 2018.
- Mikhail Belkin and Partha Niyogi. Towards a theoretical foundation for laplacian-based manifold methods. *Journal of Computer and System Sciences*, 74(8):1289–1308, 2008.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2024. URL <https://arxiv.org/abs/2410.24164>.
- Zixuan Chen, Xialin He, Yen-Jen Wang, Qiayuan Liao, Yanjie Ze, Zhongyu Li, S. Shankar Sastry, Jiajun Wu, Koushil Sreenath, Saurabh Gupta, and Xue Bin Peng. Learning smooth humanoid locomotion through lipschitz-constrained policies. *arxiv preprint arXiv:2410.11825*, 2024.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2024.
- Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. doi: 10.1090/cbms/092.
- Michael Jae-Yoon Chung, Maxwell Forbes, Maya Cakmak, and Rajesh PN Rao. Accelerating imitation learning through crowdsourcing. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4777–4784. IEEE, 2014.
- Norman Di Palo and Edward Johns. Keypoint action tokens enable in-context imitation learning in robotics. In *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- Maximilian Du, Suraj Nair, Dorsa Sadigh, and Chelsea Finn. Behavior retrieval: Few-shot imitation learning by querying unlabeled datasets. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning, 2020.
- Letian Fu, Huang Huang, Gaurav Datta, Lawrence Yunliang Chen, William Chung-Ho Panitch, Fangchen Liu, Hui Li, and Ken Goldberg. In-context imitation learning via next-token prediction. *arXiv preprint arXiv:2408.15980*, 2024.
- Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha Srinivasa, and Abhishek Gupta. Ccil: Continuity-based data augmentation for corrective imitation learning. In *The Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=LQ6LQ8f4y8>.
- Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha S. Srinivasa, and Abhishek Gupta. CCIL: continuity-based data augmentation for corrective imitation learning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024b. URL <https://openreview.net/forum?id=LQ6LQ8f4y8>.
- Taisuke Kobayashi. L2c2: Locally lipschitz continuous constraint towards stable and smooth reinforcement learning. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4032–4039. IEEE, IEEE, October 2022.

- Jonathan Lee et al. Supervised pretraining can learn in-context reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pp. 43057–43083, 2023.
- Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosior, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pp. 3744–3753, 2019.
- Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems, 2020. URL <https://arxiv.org/abs/2005.01643>.
- Li-Heng Lin, Yuchen Cui, Amber Xie, Tianyu Hua, and Dorsa Sadigh. Flowretrieval: Flow-guided data retrieval for few-shot imitation learning. In Pulkit Agrawal, Oliver Kroemer, and Wolfram Burgard (eds.), *Conference on Robot Learning, 6-9 November 2024, Munich, Germany*, volume 270 of *Proceedings of Machine Learning Research*, pp. 4084–4099. PMLR, 2024. URL <https://proceedings.mlr.press/v270/lin25a.html>.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR) 2023*, 2023. URL <https://arxiv.org/abs/2210.02747>. Originally appeared as arXiv preprint arXiv:2210.02747.
- Jiafei Lyu, Xiaoteng Ma, Le Wan, Runze Liu, Xiu Li, , and Zongqing Lu. Seabo: A simple search-based method for offline imitation learning. In *International Conference on Learning Representations*, 2024.
- Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Ireteyio Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *7th Annual Conference on Robot Learning*, 2023.
- Elman Mansimov and Kyunghyun Cho. Simple nearest neighbor policy method for continuous control tasks. In *International Conference on Learning Representations (ICLR) 2018*, 2018. Under review as a conference paper at ICLR 2018.
- Marius Memmel, Jacob Berg, Bingqing Chen, Abhishek Gupta, and Jonathan Francis. Strap: Robot sub-trajectory retrieval for augmented policy learning. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In Karen Liu, Dana Kulic, and Jeffrey Ichnowski (eds.), *Conference on Robot Learning, CoRL 2022, 14-18 December 2022, Auckland, New Zealand*, volume 205 of *Proceedings of Machine Learning Research*, pp. 892–909. PMLR, 2022. URL <https://proceedings.mlr.press/v205/nair23a.html>.
- Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.
- Jyothish Pari, Nur Muhammad Shafullah, Sridhar Pandian Arunachalam, and Lerrel Pinto. The surprising effectiveness of representation learning for visual imitation. In *Robotics: Science and Systems (RSS)*. Robotics: Science and Systems Foundation, June 2022. doi: 10.15607/RSS.2022.XVIII-052.
- Emmanuel Pignat and Sylvain Calinon. Bayesian gaussian mixture model for robotic policy imitation. *IEEE Robotics and Automation Letters*, 2019. URL <https://arxiv.org/pdf/1904.10716.pdf>. Preprint version.
- Dean Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Comput.*, 3(1):88–97, 1991. doi: 10.1162/NECO.1991.3.1.88. URL <https://doi.org/10.1162/neco.1991.3.1.88>.
- Stéphane Ross and Drew Bagnell. Efficient reductions for imitation learning. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. 661–668. JMLR Workshop and Conference Proceedings, 2010.

-
- Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In Geoffrey J. Gordon, David B. Dunson, and Miroslav Dudík (eds.), *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2011, Fort Lauderdale, USA, April 11-13, 2011*, volume 15 of *JMLR Proceedings*, pp. 627–635. JMLR.org, 2011. URL <http://proceedings.mlr.press/v15/ross11a/ross11a.pdf>.
- Steven L. Salzberg and David W. Aha. Learning to catch: Applying nearest neighbor algorithms to dynamic control tasks. In P. Cheeseman and R. W. Oldford (eds.), *Selecting Models from Data: Artificial Intelligence and Statistics IV*, volume 89 of *Lecture Notes in Statistics*, pp. 321–328. Springer, 1994. doi: 10.1007/978-1-4612-2660-4_33. URL https://link.springer.com/chapter/10.1007/978-1-4612-2660-4_33.
- Jonathan C. Spencer, Sanjiban Choudhury, Arun Venkatraman, Brian D. Ziebart, and J. Andrew Bagnell. Feedback in imitation learning: The three regimes of covariate shift. *CoRR*, abs/2102.02872, 2021. URL <https://arxiv.org/abs/2102.02872>.
- Kaustubh Sridhar, Souradeep Dutta, Dinesh Jayaraman, and Insup Lee. REGENT: A retrieval-augmented generalist agent that can act in-context in new environments. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. Oral presentation.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pp. 5026–5033. IEEE, 2012.
- Arun Venkatraman, Martial Hebert, and J. Andrew Bagnell. Improving multi-step prediction of learned time series models. In Blai Bonet and Sven Koenig (eds.), *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, January 25-30, 2015, Austin, Texas, USA*, pp. 3024–3030. AAAI Press, 2015. doi: 10.1609/AAAI.V29I1.9590. URL <https://doi.org/10.1609/aaai.v29i1.9590>.
- Jan Ole von Hartz, Adrian Röfer, Joschka Boedecker, and Abhinav Valada. The unreasonable effectiveness of discrete-time gaussian process mixtures for robot policy learning. *arXiv preprint arXiv:2505.03296*, 2025.
- Vitalis Vosylius and Edward Johns. Instant policy: In-context imitation learning via graph diffusion. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan R Salakhutdinov, and Alexander J Smola. Deep sets. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (eds.), *Advances in Neural Information Processing Systems 30*, pp. 3391–3401. Curran Associates, Inc., 2017. URL <http://papers.nips.cc/paper/6931-deep-sets.pdf>.
- Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. Learning with local and global consistency. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 16, 2003.
- Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Soroush Nasiriany, Yifeng Zhu, and Kevin Lin. robosuite: A modular simulation framework and benchmark for robot learning. In *arXiv preprint arXiv:2009.12293*, 2020.

A APPENDIX

A.1 LEMMAS AND PROOFS

A.1.1 PROOF OF THEOREM 1

To prove Theorem 1, we first start with a well-known result (Chung, 1997; Zhou et al., 2003; Belkin & Niyogi, 2008).

Lemma 1 (Smoothness regularizer as k -NN graph Laplacian penalty). *Let $\{s_1^*, \dots, s_n^*\}$ be the expert states with corresponding predicted actions $f(s_i^*) \in \mathbb{R}^{d_a}$. For each i , let $\mathcal{N}_k(s_i^*)$ denote the indices of the k -nearest neighbors of s_i^* (excluding i). Define asymmetric weights*

$$\tilde{W}_{ij} = \begin{cases} w_j(s_i^*), & \text{if } j \in \mathcal{N}_k(s_i^*), \\ 0, & \text{otherwise,} \end{cases}$$

and construct a symmetric affinity matrix

$$W_{ij} = \frac{1}{2} (\tilde{W}_{ij} + \tilde{W}_{ji}).$$

Let D be the degree matrix with $D_{ii} = \sum_j W_{ij}$, and define the k -NN graph Laplacian $L = D - W$.

Then the smoothness regularizer can be written as the quadratic form

$$\mathcal{L}_S(f) = \frac{1}{n} \sum_{i=1}^n \sum_{j \in \mathcal{N}_k(s_i^*)} w_j(s_i^*) \|f(s_i^*) - f(s_j^*)\|^2 \propto \text{Tr}(F^\top L F),$$

where $F = [f(s_1^*), f(s_2^*), \dots, f(s_n^*)]^\top \in \mathbb{R}^{n \times d_a}$. Equivalently, in the scalar case,

$$\mathcal{L}_S(f) \propto f^\top L f.$$

Corollary 1 (Continuum limit of smoothness regularizer). *Assume states $\{s_i^*\}_{i=1}^n$ are sampled i.i.d. from a smooth density $p(s^*)$ supported on an m -dimensional C^2 manifold $\mathcal{M} \subset \mathbb{R}^d$. Let W be the symmetrized k -NN affinity matrix constructed from a kernel K_Δ with bandwidth h , and let $L = D - W$ be the graph Laplacian.*

If $n \rightarrow \infty$, $h \rightarrow 0$, and $nh^{m+2} \rightarrow \infty$, then the normalized quadratic form converges to the weighted Dirichlet energy:

$$\frac{1}{n^2 h^{m+2}} \text{Tr}(F^\top L F) \rightarrow C_K \int_{\mathcal{M}} \|\nabla_{\mathcal{M}} f(s)\|_2^2 p(s)^2 d\text{vol}(s),$$

where $C_K > 0$ is a constant depending only on the kernel K_Δ .

Theorem 1 (Manifold Regularized BC ($\mathcal{L}_{\text{MRIL}}$) improves over vanilla BC ($\mathcal{L}_{\text{BC}}$)). *Let $f^* : \mathcal{S} \rightarrow \mathcal{A}$ be the true, underlying expert policy, assumed to be C^2 -smooth on a compact state space $\mathcal{S}$. Let $f : \mathcal{S} \rightarrow \mathcal{A}$ denote the learned policy estimator. Consider two estimators trained on expert demonstrations:*

1. **Vanilla BC:** a global supervised model minimizing

$$\mathcal{L}_{\text{BC}}(f) = \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)].$$

2. **MRIL:** a neighbor-based estimator minimizing

$$\mathcal{L}_{\text{MRIL}}(f) = \mathcal{L}_{\text{BC}}(f) + \lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right],$$

where $w_i(s^*)$ are the kernel weights defined above and $\lambda > 0$.

Then, under the smoothness assumption on f , the following hold:

- (i) Variance reduction: The Laplacian penalty in MRIL acts as a data-dependent Tikhonov regularizer, yielding smaller estimator variance than vanilla BC.

(ii) Smoothness guarantee: Minimizers of $\mathcal{L}_{\text{MRIL}}$ satisfy a uniform bound on the local Lipschitz constant of f , whereas vanilla BC admits interpolants with arbitrarily large Lipschitz constants between training states.

(iii) Policy stability: In a closed-loop rollout, the deviation recursion

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|^4$$

accumulates error linearly for vanilla BC, but sublinearly for MRIL, since the smoothness regularizer enforces $\|f(s^*) - f(s'^*)\| = O(\|s^* - s'^*\|)$ for neighbors s^*, s'^* .

This suggests that MRIL enjoys strictly better generalization and stability guarantees than BC.

Proof. We prove each claim in turn.

(i) Variance reduction. The Laplacian penalty in $\mathcal{L}_{\text{MRIL}}$ is

$$\sum_{i,j} W_{ij} \|f(s_i^*) - f(s_j^*)\|^2 = 2f^\top Lf,$$

where $L = D - W$ is the graph Laplacian, and W is the k -NN affinity matrix. By Lemma 1, this equals the empirical Dirichlet energy of f on the k -NN graph. It is well known (Zhou et al., 2003; Belkin & Niyogi, 2008) that such a quadratic penalty is equivalent to Tikhonov regularization with respect to the graph Laplacian norm $\|f\|_L^2 = f^\top Lf$. In statistical learning theory, adding a Tikhonov penalty strictly reduces the variance of the estimator compared to the unregularized solution while keeping the bias term controlled. Thus MRIL enjoys smaller estimator variance than vanilla BC, which uses no such penalty.

(ii) Smoothness guarantee. Consider the continuum limit (Corollary 1): for i.i.d. samples $\{s_i^*\}$ from density p on a smooth manifold $\mathcal{M}$, the normalized penalty converges to

$$\int_{\mathcal{M}} \|\nabla f(s)\|^2 p(s)^2 d\text{vol}(s).$$

This is the weighted Dirichlet energy of f on $\mathcal{M}$. If this integral is finite, f belongs to the Sobolev space $H^1(\mathcal{M}, p^2)$, and in particular f is locally Lipschitz almost everywhere with

$$\|f(s^*) - f(s'^*)\| \leq C\|s^* - s'^*\| \quad \text{for } p\text{-a.e. neighbor pairs } s^*, s'^*.$$

Therefore minimizers of $\mathcal{L}_{\text{MRIL}}$ have uniformly bounded local Lipschitz constants along high-density regions of the state space. By contrast, minimizers of vanilla BC have no such constraint: any oscillatory interpolant that matches the training data exactly yields the same supervised risk, so arbitrarily large Lipschitz constants are possible.

(iii) Policy stability. Let $\Delta_t = \|s_t - s_t^*\|$ denote the deviation at time t . For Lipschitz dynamics T ,

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|.$$

Decompose the action error:

$$\|f(s_t) - f^*(s_t^*)\| \leq \|f(s_t) - f^*(s_t)\| + \|f^*(s_t) - f^*(s_t^*)\|.$$

For vanilla BC, the first term $\|f(s_t) - f^*(s_t)\|$ is only minimized on the empirical distribution $P_{\mathcal{S}}$; off-distribution, it may be $O(1)$ regardless of Δ_t . The second term satisfies $\|f^*(s_t) - f^*(s_t^*)\| = O(\Delta_t)$ by smoothness of f^* . Hence the recursion can take the form

$$\Delta_{t+1} \leq L_s \Delta_t + L_a (O(1) + O(\Delta_t)),$$

which accumulates linearly in t .

For MRIL, the Laplacian penalty enforces

$$\|f(s^*) - f(s'^*)\| \leq C\|s^* - s'^*\| \quad \text{for neighbor pairs } (s^*, s'^*),$$

⁴ L_s and L_a are the Lipschitz constants of the environment transition dynamics with respect to state and action, respectively.

as shown in part (ii). Thus $\|f(s_t) - f^*(s_t)\| = O(h^2)$ (where h is the radius of the kernel bandwidth around the point s_t) by local-linear regression error bounds, and $\|f^*(s_t) - f^*(s_t^*)\| = O(\Delta_t)$ by smoothness of f^* . Combining these,

$$\Delta_{t+1} \leq L_s \Delta_t + L_a (O(h^2) + O(\Delta_t)).$$

Since the constant multiplying Δ_t is strictly smaller under the smoothness constraint, the cumulative error grows strictly slower than in vanilla BC. In particular, error growth is sublinear in the rollout horizon when h is small, whereas it is linear for vanilla BC.

Conclusion. Claims (i)–(iii) establish that MRIL yields lower variance, uniform smoothness control, and sublinear rollout error accumulation compared to vanilla BC, completing the proof. $\square$

Kernel choice. For the IC smoothness regularizer, we adopt a Gaussian kernel

$$w_i(s^*) \propto \exp\left(-\frac{\|s^* - s_i^*\|^2}{2h^2}\right), \quad \sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) = 1,$$

with bandwidth h set to the median distance to the k -th nearest neighbor across the dataset. This choice is standard in manifold regularization (Belkin & Niyogi, 2008; Zhou et al., 2003) and ensures that the graph Laplacian penalty converges to the Dirichlet energy in the continuum limit. In practice, we found this default to be stable across tasks, though other kernels (e.g., uniform k -NN or exponential decay) yield qualitatively similar results.

A.1.2 PROOF OF THEOREM 2

We begin with the required Lemmas establishing the spectral form of explicit Laplacian regularization and neighbor aggregation.

Lemma 2 (Spectral form of explicit Laplacian regularization). *Let L be the symmetric normalized graph Laplacian with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$. The minimizer of the penalized objective*

$$\mathcal{L}_\lambda(f) = \|f - a^*\|^2 + \lambda f^\top L f$$

has the closed-form expansion

$$f_\lambda = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j.$$

Thus λ directly determines the spectral filter $\phi_\lambda(\mu) = (1 + \lambda \mu)^{-1}$ applied to each Laplacian mode.

Proof. Diagonalize $L = U \Lambda U^\top$ with $U = [u_1, \dots, u_n]$ orthogonal and $\Lambda = \text{diag}(\mu_1, \dots, \mu_n)$. Write $f = U c$, $a^* = U b$ in this basis. The objective becomes

$$\|U c - U b\|^2 + \lambda c^\top \Lambda c = \sum_{j=1}^n (c_j - b_j)^2 + \lambda \mu_j c_j^2.$$

Minimizing each term yields $c_j = \frac{1}{1 + \lambda \mu_j} b_j$. Transforming back gives the stated expansion. $\square$

Lemma 3 (Spectral form of neighbor aggregation). *Let $S = D^{-1}A$ be the random-walk matrix of the k -NN graph, with adjacency A and degree D . For any prediction vector f , the neighbor-averaged prediction is $\hat{f} = S f$. In the Laplacian eigenbasis, this corresponds to the spectral filter*

$$\hat{f} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j,$$

i.e. $\phi_{\text{DARP}}(\mu) = 1 - \mu$.

Proof. By definition, $L = I - D^{-1/2} A D^{-1/2}$ and $S = D^{-1} A = I - L_{\text{rw}}$ where $L_{\text{rw}} = D^{-1} L$ is the random-walk Laplacian. Since L_{rw} and L share the same spectrum up to similarity transform, the eigenbasis $\{u_j\}$ diagonalizes S . Thus for each mode u_j , $S u_j = (1 - \mu_j) u_j$, yielding the claimed spectral filter. $\square$

Theorem 2 (iMRIL is parameter-free Laplacian regularization for BC (MRIL)). Consider the symmetric normalized k -NN graph Laplacian L (defined in Section 2.2), with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$.

The minimizers of the explicit MRIL objective (Section 2.2) and the implicit iMRIL objective (Section 2.3) have the following closed form expansions

$$f_{\text{MRIL}} = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j \quad \hat{f}_{\text{iMRIL}} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j$$

iMRIL's neighbor aggregation step applies the fixed spectral filter $\phi_{\text{iMRIL}}(\mu) = 1 - \mu$ to the graph Laplacian L , preserving low-frequency modes and suppressing high-frequency modes. The congruence between $\hat{f}_{\text{iMRIL}}$ and f_{MRIL} shows that iMRIL is equivalent to a built-in form of Laplacian smoothing (MRIL) with effective $\lambda \approx 1$ in normalized units. Unlike explicit regularization, this implicit filter requires no additional hyperparameter tuning.

Proof. From Lemma 3, the neighbor aggregation operator $S = D^{-1}A$ acts on Laplacian eigenmodes u_j as

$$Su_j = (1 - \mu_j)u_j,$$

where μ_j are the normalized Laplacian eigenvalues. Thus in the graph Fourier basis, neighbor aggregation corresponds to multiplying each mode by the fixed spectral filter $\phi_{\text{DARP}}(\mu) = 1 - \mu$.

On the other hand, Lemma 2 shows that explicit Laplacian regularization with parameter λ yields the spectral filter $\phi_\lambda(\mu) = (1 + \lambda \mu)^{-1}$. Both filters downweight high-frequency modes ($\mu \gg 0$) while preserving low-frequency modes ($\mu \approx 0$). The key difference is that $\phi_\lambda(\mu)$ requires tuning λ , whereas $\phi_{\text{DARP}}(\mu)$ is parameter-free.

To see the equivalence, note that for small μ ,

$$\phi_{\text{DARP}}(\mu) = 1 - \mu \approx (1 + \mu)^{-1} = \phi_{\lambda=1}(\mu) \quad \text{up to } O(\mu^2) \text{ terms.}$$

Thus DARP can be interpreted as performing Laplacian smoothing with an effective regularization weight of order $\lambda \approx 1$ in normalized units. Moreover, for large μ , $\phi_{\text{DARP}}(\mu)$ damps high-frequency modes even more strongly by driving them toward zero, providing a sharper low-pass effect than explicit regularization.

Therefore, DARP's aggregation step is mathematically equivalent to implicit Laplacian regularization with fixed spectral filter ϕ_{DARP} , eliminating the need to tune λ explicitly. $\square$

This yields the interpretation that anchors our theoretical contribution: DARP performs *locally adaptive implicit regularization*. Rather than introducing an explicit global weight λ , its neighbor

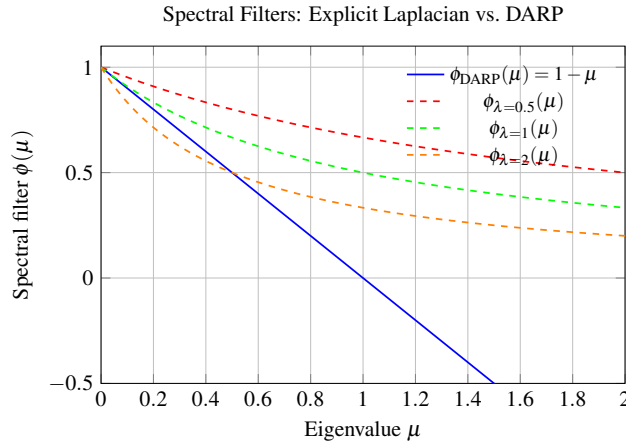


Figure 8: DARP achieves sharper low-pass filtering than explicit regularization, without tuning λ .

aggregation step enforces smoothness automatically through the graph structure, with an effective regularization strength that varies with local degree and neighborhood geometry and thus adapts to the density of the expert demonstrations. Spectrally, this is the fixed filter $\phi_{\text{DARP}}(\mu) = 1 - \mu$, which suppresses high-frequency modes more aggressively than any fixed explicit λ . Figure 8 makes the comparison visible: DARP achieves sharper low-pass filtering with no hyperparameter to tune – the precise sense in which our smoothing is “parameter-free.”

A.2 ADDITIONAL EXPERIMENTAL DETAILS

A.2.1 RETRIEVAL

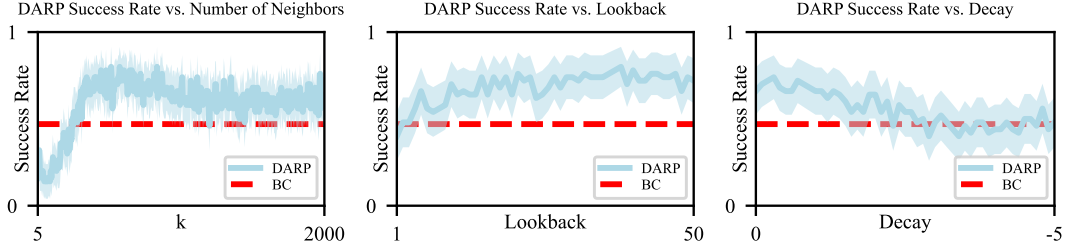


Figure 9: **How DARP responds to its retrieval hyperparameters.** (left) Performance is poor with few neighbors, peaks at about 500 retrieved neighbors, and plateaus just above BC’s success rate as k approaches the dataset size; (center) performance generally improves slightly as more history is used, dropping below BC only when little or no history is considered; (right) performance is sensitive to how much older observations are down-weighted – if the decay is too aggressive, DARP behaves as if it had almost no lookback and falls below BC. Success rate is over 50 trials on the Robosuite Stack environment, with 95% confidence intervals.

The choice of distance function $d(s_q, s_i^*)$ used to select the k neighbors is a crucial design decision for DARP. While plain Euclidean distance between states already works, our experiments use a slightly modified distance that exploits the sequential nature of the data by incorporating history into the calculation.

Suppose we have a query trajectory $S_q = (\dots, s_{q,-1}, s_{q,0})$ where $s_{q,0}$ is the current query state s_q . Now suppose we want to calculate $d(s_q, s_i^*)$, where s_i^* is some state from the expert dataset. We first find the trajectory this state is from—call this S_j^* —and the index of s_i^* in this trajectory—call this t . Thus, s_i^* can be rewritten as $s_{j,t}^*$. Given some lookback parameter ℓ which denotes how many past states we want to consider, we get:

$$d(s_q, s_i^*) = \sum_{n=0}^{\ell-1} \|s_{q,-n} - s_{j,t-n}^*\|$$

This is simply the accumulation of Euclidean distances of the current and last $\ell - 1$ states from both the query trajectory and the source trajectory, assuming valid indices. Of course, in practice, we generally want to put more emphasis on more recent states, as we want them to be more influential in the selection of neighbors. Thus, given some rate of exponential decay $\gamma \geq 0$, we have

$$d(s_q, s_i^*) = \sum_{n=0}^{\ell-1} \|s_{q,-n} - s_{j,t-n}^*\| \cdot e^{-\gamma n}$$

See Figure 9 for an experimental analysis on how the success rate in an environment changes as these parameters are swept.

A.2.2 CAN DARP HANDLE TASKS REQUIRING THE REPRESENTATION OF MULTI-MODAL ACTION DISTRIBUTIONS?

This experiment supports the generality half of our contribution by testing DARP on complex action distributions. We use the Push-T task of (Chi et al., 2024), which demands representing multi-modal action distributions (Figure 10), and equip DARP with a Set Transformer head that predicts the parameters of a Gaussian Mixture Model. DARP with this GMM head handles multimodality effectively, improving over BC by 22% on Push-T (Q2) (Table 4). This confirms that the parametric-aggregator design extends DARP to multi-modal action modeling.

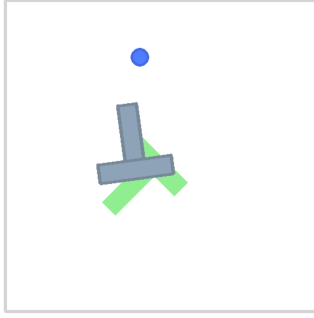


Figure 10: **Push-T Environment.** The goal is to control the blue circle to push the T-shaped block.

Method	Score
BC	48 ± 8
DARP	70 ± 8

Table 4: **Push-T results.** Averaged over 100 trials, DARP outperforms BC on this multimodal task.

A.2.3 CAN DARP HANDLE DISCONTINUOUS ENVIRONMENTS WHERE NEARBY STATES MAY REQUIRE OPPOSING ACTIONS?

A natural objection to any neighbor-based approach is that it might fail under strong discontinuities, where states close in Euclidean distance require drastically different actions. We confront this objection head-on with a stress test built on a modified D4RL Umaze environment (Figure 11).

Even in this deliberately adversarial discontinuous environment, DARP reaches a 57% success rate versus BC’s 25% (Q3; Table 5). This is further evidence for the ablation’s conclusion: the difference vectors and permutation-invariant aggregation let DARP tell appropriate neighbors from inappropriate ones even when spatial proximity does not imply action similarity.

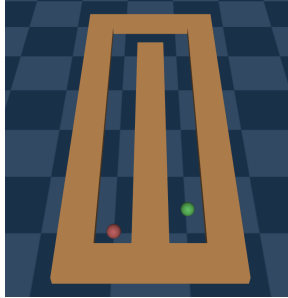


Figure 11: **Long maze environment.** The goal is to move a force-actuated ball from the green start to the red destination.

Method	Succ. (%)
BC	25
DARP	57

Table 5: **Long-maze results.** Averaged over 100 trials, DARP significantly outperforms BC even under strong discontinuities.

A.2.4 CAN DARP RECOVER FROM BC ERROR?

To probe DARP’s robustness to accumulated error directly, we roll out a BC agent in an environment where it is known to fail and, every k steps, fork the environment and launch a DARP agent from that clone. The results (Fig. 12) are clear: even as BC drifts away from the support of the expert demonstrations, DARP recovers and scores very highly. This is direct evidence for the robustness that underlies our central contribution – DARP performs well in exactly the slightly out-of-distribution states that a failing BC agent drifts into.

A.2.5 COMPARISON WITH CCIL

To position our implicit-smoothing contribution against explicit smoothing, we compare DARP with CCIL, a baseline that explicitly induces smoothness (Table 6). Using the scores reported in the CCIL

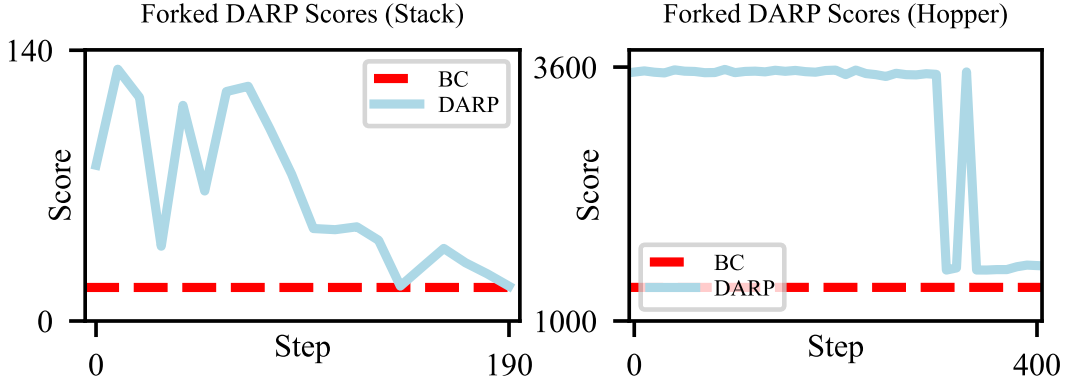


Figure 12: **DARP recovers from states a failing BC agent drifts into.** On two tasks (Robosuite Stack and MuJoCo Hopper), we roll out a BC agent and fork the environment every k steps ($k = 10$). Even as BC nears the end of its failing rollout, DARP still scores highly when launched from those states – losing this ability only about halfway through the Stack rollout and about 80% of the way through the Hopper rollout.

Environment	Method	Improvement over BC (%)
Hopper	CCIL	32.6%
	DARP	53.2%
Walker	CCIL	84.1%
	DARP	84.1%
Ant	CCIL	12.6%
	DARP	84.5%
HalfCheetah	CCIL	5.4%
	DARP	418.7%

Table 6: **DARP matches or exceeds CCIL’s relative gain over BC on every task.** Relative improvement over BC for DARP versus CCIL on standard MuJoCo benchmarks, showing our implicit-smoothing contribution is competitive with an explicit-smoothing baseline.

paper and comparing percent improvement over BC, DARP significantly outperforms CCIL on three of four environments, with an especially wide margin on HalfCheetah (418.7% vs. 5.4%).

A.2.6 DISTANCE METRIC SENSITIVITY

Method	Success Rate (%)
BC	47
DARP (Cosine Similarity)	70
DARP (Euclidean Distance)	75

Table 7: **DARP is robust to the choice of distance metric.** Success rates using R3M features on the Robosuite Stack task: Euclidean distance and cosine similarity perform similarly, beating BC by 28% and 23% respectively.

Although all experiments in this paper select neighbors by Euclidean distance, alternative metrics such as cosine similarity are natural to consider, especially in high-dimensional embeddings like R3M. Table 7 shows that DARP performs similarly – only 5% worse – under cosine similarity, indicating that the contribution does not hinge on a particular distance metric.

Method	Stack	Threading	Peg Insertion
BC (MLP)	34	12	28
DARP (MLP)	62	38	40
Diffusion	52	44	54
DARP w/ Diffusion	82	72	66

Table 8: **DARP composes with diffusion policy for further gains.** DARP improves both standard MLP and diffusion policies on the Robosuite Stack, Threading, and Peg Insertion tasks (success rate over 50 trials, state-based). This shows the contribution is orthogonal to the backbone rather than tied to MLPs. Fewer demonstrations are used than in Table 2, so the baseline BC and DARP numbers differ from that table.

A.2.7 DARP IN COMBINATION WITH DIFFUSION POLICY

Our main experiments use an MLP backbone, but diffusion policy (Chi et al., 2024) is a state-of-the-art model class for imitation learning, particularly for manipulation. Table 8 makes an important point about the scope of the contribution: DARP is not an alternative to diffusion but composes with it. Pairing the DARP architecture with a diffusion backbone beats DARP with an MLP backbone by 20%, 34%, and 26% success rate on the three tasks, and beats standard BC by a total of 48%, 60%, and 38% respectively.

Method	Training (s/epoch)	Inference (s/step)
Diffusion	5.610	0.15700
DARP (MLP)	0.559	0.00437

Table 9: **DARP (MLP) is far cheaper than diffusion policy.** Runtime costs of DARP (MLP) versus diffusion policy: diffusion is $\approx 10\times$ slower to train and $\approx 36\times$ slower at inference, underscoring that DARP delivers its gains without a heavyweight backbone.

Complementing this, Table 9 shows that DARP with an MLP backbone is much faster than standard diffusion, especially at inference – so the gains reported throughout the paper come at a modest computational cost.

A.2.8 DARP TRAINED ON HUMAN DEMONSTRATIONS

Method	Success Rate (%)
BC	45
DARP	60

Table 10: **The gain persists with human demonstrations.** Success rates when training on human data rather than data collected by RL policies, on the Robosuite Stack task.

Because our main MuJoCo and Robosuite demonstrations come from an optimal reinforcement learning policy, a fair test of the contribution’s generality is whether the gain survives with genuinely human data. It does: on the Robosuite Stack task with human demonstrations, DARP outperforms standard BC by 15% (Table 10).

A.2.9 CHOICE OF RETRIEVAL HYPERPARAMETERS

Environment	Method	Score (Mean)
Hopper	BC	507.49
	DARP	805.69
Stack	BC	0.12
	DARP	0.31

Table 11: **DARP keeps its edge over BC when hyperparameters are chosen by validation loss.** Mean scores on the Hopper and Stack environments, with training epochs and retrieval hyperparameters selected by validation loss.

Figure 9 shows that some retrieval settings (e.g., very few neighbors) can make DARP underperform BC, so the gains are not automatic. Reassuringly for practitioners, selecting retrieval hyperparameters by minimizing validation loss is an effective, principled way to land on performant settings (Table 11).

A.3 PSEUDOCODE

For reproducibility, Algorithm 1 gives complete pseudocode for the DARP training procedure.

Algorithm 1 Difference-Aware Retrieval Policies

```

1: Input: Expert demonstrations  $\mathcal{D}^* = \{(s_j^*, a_j^*)\}$ , number of neighbors  $k$ 
2: Initialize:  $f$  parameters  $\theta$ 
3: if  $g$  is parametric then
4:   Initialize:  $g$  parameters  $\psi$ 
5: end if
6: // Training Loop
7: while not converged do
8:   Sample batch of query data  $(s_q^*, a_q^*) \sim \mathcal{D}^*$ 
9:   for each query pair  $(s_q^*, a_q^*)$  in batch do
10:    // Find  $k$ -Nearest Neighbors from the entire dataset  $\mathcal{D}^*$ 
11:     $\mathcal{N}_k(s_q^*) \leftarrow \text{argmin-}k; d(s_q^*, s_j^*)$ 
12:    // Compute Neighbor-based Predictions
13:    for each neighbor index  $i \in \mathcal{N}_k(s_q^*)$  do
14:       $a'_i \leftarrow f_\theta(s_i^*, a_i^*, s_i^* - s_q^*)$ 
15:    end for
16:    // Aggregate Predictions
17:    if  $g$  is parametric then
18:       $\hat{a}_q \leftarrow g_\psi(\{a'_i\}_{i \in \mathcal{N}_k(s_q^*)})$ 
19:    else
20:       $\hat{a}_q \leftarrow g(\{a'_i\}_{i \in \mathcal{N}_k(s_q^*)})$ 
21:    end if
22:  end for
23:  // Update Parameters based on the batch loss
24:   $\mathcal{L} \leftarrow \sum_{(s_q^*, a_q^*) \in \text{batch}} \|\hat{a}_q - a_q^*\|^2$ 
25:  // Gradient descent step
26:   $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}$ 
27:  if  $g$  is parametric then
28:     $\psi \leftarrow \psi - \alpha \nabla_\psi \mathcal{L}$ 
29:  end if
30: end while
31: Output: Trained parameters  $\theta$  and, if applicable,  $\psi$ 

```

A.4 RUNTIME ANALYSIS

Environment	k	Train (s/epoch)	Test (s/step)
Hopper (1,000 datapoints)	BC	0.102	0.00127
	100	0.126	0.00413
	200	0.128	0.00418
	300	0.130	0.00420
	400	0.131	0.00421
	500	0.130	0.00419
Stack (4,200 datapoints)	BC	0.431	0.00139
	100	0.556	0.00437
	250	0.539	0.00442
	500	0.559	0.00437
	750	0.576	0.00433
	1000	0.591	0.00478
	1500	0.682	0.00452
	2000	0.758	0.00451

Table 12: **DARP’s runtime overhead over BC is small and scales sub-linearly in k .** Training and testing speeds (seconds) for Behavior Cloning (BC) and varying k on the Hopper and Stack datasets.

Table 12 analyzes DARP’s computational overhead at both training and inference time, addressing a practical concern about retrieval-based policies. Two findings matter for deployment: computation scales sub-linearly in k , and inference is fast enough for real-time robotics. On the Robosuite Stack task at $k = 500$, for instance, inference takes about 0.00437 seconds per step on our hardware – a control frequency above 230 Hz.